Are Proof Structures Machine-Learnable? A Study with DAG-Structured Proof Terms

Christoph Wernhard[1] and Zsolt Zombori[2,3]

[1] University of Potsdam, Germany
[2] HUN-REN Alfréd Rényi Institute of Mathematics, Hungary
[3] KU Leuven, Belgium

Addressed Questions and Our Approach via Proof Terms. We investigate the generation of mathematical proofs using neural language models. Specifically, we address the following questions:

Q1. Can a language model learn the syntax of proof terms?
Q2. Can it capture sufficient semantic constraints to ensure that proof terms indeed prove a most general theorem formula, i.e., in the jargon of type theory, are well-typed?
Q3. Can it generate proofs of mathematically interesting statements?
Q4. Can generated proofs yield useful lemmas for automated reasoning?

Proofs are often studied as derivations, emphasizing the sequence of inference steps that lead to a conclusion. In contrast, we adopt a static perspective, representing proofs as structured objects – namely, terms with a representation as directed acyclic graph (DAG). This view follows a well-established line of work, rooted in Meredith's condensed detachment for investigating axiomatizations of logics [7, 8, 6], and continued notably in type theory ("formulas as types"), e.g., [1], as well as in automated reasoning, e.g., [2, 9, 12].

Representing proofs as terms allows us to give emphasis to their structure, abstracting from the way in which proofs were derived. We do no need to distinguish between different derivations, if these are just different ways to construct the same proof term. This choice of proof representation has important implications for the learning task. When proofs are represented as sequences of derivation steps, learning methods naturally capture patterns in the underlying proof search process – for example, heuristics for clause selection or rule application. In contrast, our representation focuses solely on the resulting proof object. While this may limit insight into proof search decisions, it enables the identification of global structural regularities within proofs.

With respect to proof generation, our ambition is twofold. First, we investigate whether a language model can generate proofs that solve known non-trivial problems (Q1–Q3). Second, we explore the use of proofs generated with a trained model to obtain lemmas for improving proof search with conventional symbolic first-order provers (Q4).

Synthetic Proofs for Pre-Training and Human-Made Proofs for Fine-Tuning. Since proof terms are defined as a formal language, proof generation can be cast as a language generation task. Exploiting the widespread success of neural language models, we use standard state-of-the-art neural language models and training algorithms to train a proof generator. Because our proof representation constitutes a highly specialized formal language that is not present in standard training corpora, we expect limited benefit from existing pre-trained models. For this reason, we train our model from scratch. Our neural model is a transformer architecture with around 130 million parameters. We represent proof terms by the corresponding minimal DAG, serialized as a DAG-grammar.

Specifically, we consider proof terms built from the binary function symbol D, representing application of the inference rule *condensed detachment*, and constants, representing axioms. As

an example, consider the proof term

$$\mathsf{D}(\mathsf{D}(\texttt{ax-1},\texttt{ax-1}),\mathsf{D}(\mathsf{D}(\texttt{ax-1},\mathsf{D}(\texttt{ax-1},\texttt{ax-1})),\mathsf{D}(\texttt{ax-1},\mathsf{D}(\texttt{ax-1},\texttt{ax-1})))),$$

which can be represented by the DAG grammar

$$\{2 \to \mathsf{D}(\texttt{ax-1},\texttt{ax-1}),\ 3 \to \mathsf{D}(\texttt{ax-1},2),\ \texttt{start} \to \mathsf{D}(2,\mathsf{D}(3,3))\}.$$

Here `ax-1` is a constant that identifies an axiom and $2, 3, \texttt{start}$ are nonterminals. A proof term provides a proof of all substitution instances of its *most general theorem*, a formula that can be calculated by means of unification on the basis of axiom formulas associated with the axiom identifiers. If the involved unification fails, the proof term is said to be not well-typed and its most general theorem is undefined. For training the language model, proof terms are represented by the corresponding minimal DAG, serialized as a DAG grammar with terms written in parentheses-free Polish notation, e.g., `D(X1=da1a1|X2=da1X1|s=dX1dX2X2)` for our example proof term.

Training consists of two phases. First, we train on a large corpus of small synthetic proofs. The expected key outcome from this phase is to learn the syntax and the basic properties that make an expression a well-typed proof term. In the second phase, we fine-tune the model with a small set of human-made formal proofs, aiming to focus the pre-trained model's general ability to generate proofs towards proofs of practical interest. For the synthetic corpus, we use the *SGCD* prover [10, 11] in configurations for straightforward heuristically restricted enumeration of proofs, obtaining a set of around 51 million short proofs. Our source of human proofs is *Metamath* [4, 3, 5], a system for archiving, verifying, and studying mathematical proofs, with its largest database *Metamath Proof Explorer*, also known as *set.mm*. We consider the axiomatization of propositional logic in *set.mm* with three axioms, and extract from the best-curated two-thirds of *set.mm* all those 1,374 theorems that depend on no other axioms, along with their proofs [11]. This set of theorems serves as a benchmark, with subsets designated for roles such as training and testing, or varying levels of difficulty for specific symbolic systems.

Experiments and Findings. Using synthetic and human proofs as datasets, we train a language model and investigate it at different stages through experiments refining the initially stated general questions. Questions Q1 and Q2 are now specialized to our language model:

- E1. **Can our language model learn the syntax of proof terms?** Our answer is a confident yes. Large-scale pre-training is essential for learning syntax and well-typedness.
- E2. **Can our language model learn to generate well-typed proof terms?** This is a much harder task, but our answer is that it mostly can.

Question Q3 – generating proofs of mathematically interesting statements – is a challenging long-term task. We consider two particular aspect in our experiments:

- E3.1 **Can our language model learn to generate complex proofs?** With pre-training, the model only learns to generate very small proofs. Fine-tuning on longer and more complex proofs progressively yields more and more complex and non-trivial proofs.
- E3.2 **Can our language model generalize from training with proofs of benchmark theorems to other proofs of benchmark theorems?** We see this capability emerging with fine-tuning, although this happens rarely, in around 2% of the generated samples. We also see that over time, harder and harder problems are proven by the model.

Finally, we address Q4, usefulness as lemmas for ATP systems, now with a concrete benchmark:

E.4 **Are the proofs generated by the language model identifying theorems that are useful as lemmas for proving benchmark problems?** This varies across provers and depends on how we select lemmas from the generated proofs. In one sequence of experiments, we select the 10 generated lemmas whose most general theorem has the smallest term size. Some systems benefit substantially: we see 15–20% improvement for *Vampire* and up to 140% improvement for *leanCoP* with respect to the number of problems solved across different time limits. In contrast, the performance of *E* decreases slightly.

Summary. The main contributions of our work can be summarized as follows.

1. We present proof generation as language modeling over DAG-structured proof terms.
2. We demonstrate that neural models can learn syntactic correctness and approximate well-typedness of proof terms, giving a definitively positive answer to Q1, and a mostly positive answer to Q2.
3. We show that fine-tuning enables generation of non-trivial proofs that are new, i.e., were not seen during training. This turns out as an important step towards addressing Q3.
4. We empirically evaluate the usefulness of generated lemmas across multiple ATP systems, giving a positive answer to Q4.

Acknowledgments. Funded by the Deutsche Forschungsgemeinschaft (DFG, German Research Foundation) – Project-ID 457292495, the Hungarian Artificial Intelligence National Laboratory (RRF-2.3.1-21-2022-00004) and the European Union (ERC, DeepLog, 101142702).

References

[1] Hindley, J.R., Meredith, D.: Principal type-schemes and condensed detachment. Journal of Symbolic Logic **55**(1), 90–105 (1990). https://doi.org/10.2307/2274956

[2] McCune, W., Wos, L.: Experiments in automated deduction with condensed detachment. In: Kapur, D. (ed.) CADE-11. LNCS (LNAI), vol. 607, pp. 209–223. Springer (1992). https://doi.org/10.1007/3-540-55602-8_167

[3] Megill, N., Wheeler, D.A.: Metamath: A Computer Language for Mathematical Proofs. lulu.com, second edn. (2019), online https://us.metamath.org/downloads/metamath.pdf

[4] Megill, N.D.: A finitely axiomatized formalization of predicate calculus with equality. Notre Dame J. of Formal Logic **36**(3), 435–453 (1995). https://doi.org/10.1305/ndjfl/1040149359

[5] Megill, N.D.: Home Page – Metamath (2026), online: https://us.metamath.org/, accessed Apr 13, 2026

[6] Meredith, C.A., Prior, A.N.: Notes on the axiomatics of the propositional calculus. Notre Dame J. of Formal Logic **4**(3), 171–187 (1963). https://doi.org/10.1305/ndjfl/1093957574

[7] Prior, A.N.: Logicians at play; or Syll, Simp and Hilbert. Australasian Journal of Philosophy **34**(3), 182–192 (1956). https://doi.org/10.1080/00048405685200181

[8] Prior, A.N.: Formal Logic. Clarendon Press, Oxford, 2nd edn. (1962). https://doi.org/10.1093/acprof:oso/9780198241560.001.0001

[9] Ulrich, D.: A legacy recalled and a tradition continued. J. Autom. Reasoning **27**(2), 97–122 (2001). https://doi.org/10.1023/A:1010683508225

[10] Wernhard, C.: Structure-generating first-order theorem proving. In: Otten, J., Bibel, W. (eds.) AReCCa 2023. CEUR Workshop Proc., vol. 3613, pp. 64–83. CEUR-WS.org (2024), https://ceur-ws.org/Vol-3613/AReCCa2023_paper5.pdf

[11] Wernhard, C.: Generating theorems by generating proof structures. In: Biere, A., Lutz, C., Negri, S. (eds.) IJCAR 2026. LNCS (LNAI), Springer (2026), to appear; preprint https://arxiv.org/abs/2602.15511

[12] Wernhard, C., Bibel, W.: Investigations into proof structures. J. Autom. Reasoning **68**(24) (2024). https://doi.org/10.1007/s10817-024-09711-8